

Analisis Program Sistem Rute Bus



Oleh:

Dzikra Alghufroon (251524041)

Muhammad Iqbal (251524057)

Nadhief Musyaffa (251524059)

Deskripsi Masalah

Program ini mencari rute terpendek (minimal jumlah halte) dalam jaringan bus Trans Padang yang terdiri dari 3 koridor. Input adalah nama halte keberangkatan dan tujuan; output adalah urutan halte yang dilalui dan estimasi waktu tempuh.

Pada dasarnya adalah Shortest Path (unweighted) pada graf tak-berarah. BFS dipilih karena:

- BFS menjamin jalur dengan jumlah simpul minimum pada graf tak-berarah tanpa bobot.
- BFS menelusuri node level per level dari origin – simpul tujuan yang pertama kali ditemukan pasti memiliki depth terkecil.

1. Analisis Proses, Data Input dan Data Output(menjelaskan secara general tentang pencarian rute)

1.1 Analisis Proses

1. Inisialisasi Program membaca file halte.txt dan menyimpan seluruh data halte ke dalam Array of Struct Corridor di memori. Setiap koridor berisi array of string (char* halte[]) yang menyimpan nama-nama halte. Jika file tidak ditemukan, program akan menggunakan data bawaan (fallback data).
2. Tampilan Menu Program menampilkan menu utama dengan 4 pilihan: Cari Rute, Tambah Halte Baru, Hapus Halte, dan Keluar. User memilih dengan memasukkan angka 1–4.
3. Eksekusi Pilihan:
 - Cari Rute (1): User memasukkan nama halte asal dan tujuan. Program mencari kedua halte di seluruh koridor, kemudian mencetak rute dengan dua pendekatan:
 - Navigasi Array Koridor Mencari titik transit (irisan) antar koridor lalu menelusuri array halte untuk mencetak daftar halte yang dilewati.
 - Binary Search Tree (BST) Membangun pohon dari halte asal ke tujuan sebagai representasi rute, lalu mencari node tujuan menggunakan BFS (Queue). Setelah selesai, pohon dihapus dari memori.
 - Tambah Halte Baru (2): User memilih koridor lalu memasukkan nama halte baru. Program memvalidasi duplikat nama dalam koridor yang sama. Jika valid, nama halte ditambahkan ke akhir array koridor dan seluruh data disimpan kembali ke file.
 - Hapus Halte (3): User memilih koridor, melihat daftar halte, lalu memilih nomor halte yang akan dihapus. Program membebaskan memori string halte,

menggeser elemen array setelahnya ke kiri, mengurangi jumlah halte, dan menyimpan perubahan ke file.

- Keluar (4): Program berhenti.

1.2 Analisis Data Input

Program menerima data dari dua sumber:

1. Data dari File Eksternal (data/halte.txt)

- Format: File teks dengan struktur berantai:
- Baris diawali # menandai nama koridor (contoh: #Koridor I)
- Baris setelahnya (tanpa #) adalah nama-nama halte dalam koridor tersebut
- Struktur data internal: Data disimpan dalam Corridor corridors[MAX_CORRIDORS] sebuah Array of Struct. Setiap struct Corridor berisi:
 - char name[50] nama koridor
 - char* halte[MAX_STOPS_PER_CORRIDOR] array of pointers ke string nama halte (alokasi dinamis)
 - int jumlah_halte jumlah halte dalam koridor
- Fallback data: Jika file tidak ditemukan, program menggunakan data bawaan init_bus_data() dengan 3 koridor dan total ~53 halte

2. Data dari Input User (Runtime)

- Pilihan menu (angka 1–4) dibaca sebagai string lalu dikonversi ke integer dengan atoi()
- Nama halte asal & tujuan string maksimal 100 karakter (untuk fitur Cari Rute)
- Nama halte baru string maksimal 100 karakter (untuk fitur Tambah Halte)
- Pilihan koridor angka 1–N, dikonversi ke indeks 0-based
- Pilihan nomor halte angka untuk menentukan halte yang akan dihapus
- Konfirmasi hapus karakter y/n

Validasi Input:

- Koridor yang dipilih harus dalam rentang yang tersedia
- Nama halte baru tidak boleh kosong
- Nama halte baru tidak boleh duplikat dalam koridor yang sama
- Kapasitas koridor maksimal 25 halte
- Nomor halte yang akan dihapus harus dalam rentang yang tersedia

1.3 Analisis Data Output

1. Output ke Layar (Terminal)

- Tampilan menu daftar 4 pilihan yang ditampilkan setiap siklus
- Daftar koridor menampilkan nomor, nama koridor, dan jumlah halte
- Daftar halte menampilkan semua halte dalam suatu koridor dengan nomor urut
- Hasil pencarian rute:
 - Header rute dengan nama halte asal dan tujuan
 - Daftar halte yang dilewati dari asal ke tujuan (termasuk titik transit antar (koridor)
 - Informasi koridor yang dilewati (format: Koridor N)

- Titik transit ditandai dengan -- TRANSIT: Nama Halte (Koridor A --> Koridor B)
- Pesan konfirmasi:
 - Halte 'X' berhasil ditambahkan ke Y.
 - Halte berhasil dihapus.
 - Data berhasil disimpan ke 'data/halte.txt'
- Pesan error/warning:
 - Halte asal/tujuan 'X' tidak ditemukan
 - ERROR: Halte 'X' sudah ada di Y!
 - ERROR: Koridor tidak valid!
 - Nomor halte tidak valid!
 - Pilihan tidak valid!
 - Nama halte tidak boleh kosong!
- 2. Output ke File Eksternal (data/halte.txt)
 - Format sama dengan format input: baris #NamaKoridor diikuti daftar halte
 - File ditulis ulang setiap kali ada perubahan data (tambah/hapus halte)
- 3. Output Sementara (Internal)
 - Binary Search Tree dibangun di memori saat pencarian rute, langsung dihapus (destroyTree()) setelah selesai
 - Queue (array static) digunakan untuk BFS pada tree, sifatnya sementara per request rute

2. Struktur Data

1. Array of Struct

Informasi

Array of Struct adalah struktur data yang menggabungkan array (kumpulan data dengan tipe sama) dan struct (kumpulan data dengan tipe berbeda dalam satu kesatuan). Pada program ini, array of struct digunakan untuk menampung grup-grup data beserta atributnya.

```
#define MAX_GROUPS 10
```

```
typedef struct {
    char name[50];    // Nama grup
    char** items;     // Dynamic array of string pointers
    int count;        // Jumlah item terisi
    int capacity;     // Kapasitas yang sudah dialokasi
} Group;
```

```
Group groups[MAX_GROUPS];
```

Peran

- Menyimpan seluruh data master di memori selama program berjalan
- Menjembatani data dari file ke dalam program (input)

- Menyediakan data untuk ditampilkan ke user (output)
- Menampung perubahan data dari proses tambah dan hapus

Analisis Proses, Input, dan Output

- Proses: Data dari file dibaca baris per baris, kemudian disimpan ke dalam array of struct. Setiap baris # menandai grup baru, baris biasa menjadi item dalam grup.
- Input: File eksternal dengan format: #NamaGrup diikuti item-itemnya.
- Output: Data ditampilkan ke layar dalam bentuk daftar grup dan item, serta ditulis kembali ke file saat ada perubahan.

2. Dynamic Array

Informasi

Dynamic array adalah array yang ukurannya dapat berubah secara otomatis saat program berjalan. Berbeda dengan array statis yang ukurannya tetap sejak kompilasi, dynamic array dapat tumbuh (grow) saat item ditambah dan mengecil (shrink) saat item dihapus.

```
char** items; // Pointer ke array of pointers
int count;    // Jumlah terpakai
int capacity; // Kapasitas teralokasi
```

Peran

- Menyimpan daftar item (string) dalam setiap grup secara fleksibel
- Menghilangkan batas maksimal jumlah item per grup
- Mengoptimalkan penggunaan memori dengan menyesuaikan kapasitas sesuai kebutuhan

Analisis Proses, Input, dan Output

- Proses: Saat item ditambahkan, jika kapasitas penuh maka array digandakan ukurannya. Saat item dihapus dan kapasitas terlalu longgar, array dikecilkan setengahnya. Item baru disimpan di akhir array. Item yang dihapus menyebabkan item setelahnya bergeser ke kiri.
- Input: Item baru dari user (keyboard) saat proses tambah.
- Output: Daftar item ditampilkan ke layar dalam urutan sesuai array, dan ditulis ke file dengan urutan yang sama.

3. Binary Search Tree (BST)

Informasi

Binary Search Tree adalah struktur data pohon biner di mana setiap node memiliki paling banyak dua anak (kiri dan kanan). Node di kiri memiliki nilai lebih kecil dari parent, node di kanan memiliki nilai lebih besar. Setiap node menyimpan data dan pointer ke anak kiri, anak kanan, serta orangtua.

```
typedef struct Node {
    int id;           // Kunci pengurutan
    char name[100];   // Data
```

```

int flag;           // Penanda khusus
struct Node* left;  // Anak kiri
struct Node* right; // Anak kanan
struct Node* parent; // Orangtua
} Node;

```

Peran

- Membangun representasi sementara dari suatu sesi/rute berdasarkan data di array of struct
- Menyusun node secara terurut berdasarkan id untuk memudahkan pencarian
- Bersifat sementara — dibuat saat diperlukan dan dihancurkan setelah selesai digunakan

Analisis Proses, Input, dan Output

- Proses: BST dibangun dengan mengambil data dari array of struct. Node dimasukkan satu per satu menggunakan aturan BST (berdasarkan id). Setelah selesai digunakan, seluruh node dihapus dari memori.
- Input: Data dari array of struct (hasil dari proses input sebelumnya).
- Output: Struktur pohon di memori yang siap diproses lebih lanjut (misal untuk pencarian).

4. Queue (statis)

Informasi

Queue adalah struktur data yang menerapkan prinsip FIFO (First In, First Out). Elemen yang pertama masuk akan menjadi yang pertama keluar. Implementasi menggunakan array statis dengan dua pointer penanda depan dan belakang.

```

#define MAX_QUEUE 500
static Node* queue[MAX_QUEUE];

```

Peran

- Mendukung proses pencarian pada BST dengan metode BFS (Breadth-First Search)
- Menyimpan node-node yang akan dikunjungi secara berurutan
- Bersifat sementara — hanya aktif selama proses pencarian berlangsung

Analisis Proses, Input, dan Output

- Proses: Root tree dimasukkan ke queue. Selama queue tidak kosong, node depan dikeluarkan dan diperiksa. Jika cocok, proses selesai. Jika tidak, anak kiri dan kanan dari node tersebut dimasukkan ke queue. Proses diulang sampai queue kosong atau target ditemukan.
- Input: Node-node dari BST yang sudah dibangun sebelumnya.
- Output: Node tujuan jika ditemukan, atau NULL jika tidak ada dalam tree.

3. Spesifikasi Proses

Cara Kerja Sistem Pencarian Rute Bus

- A. **Penyiapan Data Halte Bus**
Sistem sudah menyimpan daftar lengkap halte bus secara permanen di dalam program. Setiap halte dibekali tiga informasi dasar: nomor pengenal khusus, nama halte, dan sebuah penand rute untuk mengenali apakah di halte tersebut penumpang perlu berganti bus (transit) atau bisa terus menggunakan bus yang sama.
- B. **Pemilihan Titik Awal dan Tujuan**
Penumpang cukup mengetikkan nama halte tempat mereka berangkat dan nama halte yang ingin mereka tuju.
- C. **Penyusunan Peta Jalur**
Setelah lokasi dipilih, program akan secara otomatis menyusun peta. Halte keberangkatan akan diletakkan di titik paling awal (sebagai pusat keberangkatan), lalu halte-halte lainnya disambungkan membentuk cabang-cabang jalan berdasarkan nomor pengenalnya.
- D. **Pencarian Rute Terbaik**
Sistem kemudian akan menyisir peta tersebut selangkah demi selangkah, menyebar dari titik awal keberangkatan. Cara ini dipastikan dapat melacak dan menemukan jalur yang paling efisien menuju halte tujuan.
- E. **Penentuan Daftar Rute dan Titik Transit**
Begitu halte tujuan ditemukan, program akan menelusuri balik jalan yang tadi sudah dilewati untuk menyusun daftar perjalanan yang utuh. Sambil menelusuri balik, program juga mengecek penanda rute pada tiap halte; jika penanda tersebut menunjukkan adanya perpindahan jalur, halte itu akan dicatat sebagai lokasi transit.
- F. **Menampilkan Panduan Perjalanan**
Pada akhir proses, layar akan menampilkan panduan perjalanan yang utuh. Panduan ini berisi daftar urutan halte yang akan dilewati penumpang dari awal sampai akhir, lengkap dengan informasi di halte mana saja mereka harus turun untuk berganti bus.

4. Abstraksi Data

4.1 Buat Setiap Halte diKoridor (waktu jadi tree)

```
typedef struct TreeNode {  
    int id; //identifier  
    char name[100]; // nama halte  
    int jalur; // bernilai 0 jika bus tidak pindah dan 1 jika pindah  
    struct TreeNode* leftchild; // array dinamis pointer ke child nodes  
    struct Treenode* RightChild;  
    struct TreeNode* parent; // parent node (untuk tracing rute)  
} TreeNode;
```

4.2 Buat algoritma BFS

```
typedef struct QueueNode {  
    TreeNode* tree_node;  
    struct QueueNode* next;  
} QueueNode;
```

```
typedef struct {  
    QueueNode* front;  
    QueueNode* rear;  
} Queue;
```

4.3 Buat Setiap Halte diKoridor (Sebelum jadi Tree)

```
typedef struct koridor {  
    char name[15];  
    string halte[21]; // list halte  
    int jumlah_halte;  
} Koridor;
```

5. ADT

6. Header

7. Main Program

Data Buat Koridor

1. Koridor 1

1. Pool Damri
2. Masjid Nurul Iman
3. RTH Imam Bonjol Padang
4. GOR H Agus Salim
5. Masjid Raya Sumatera Barat
6. DPRD Provinsi Sumatera Barat
7. Basko Grand Mall
8. Simpang Labor
9. Universitas Negeri Padang Air Tawar
10. Simpang Tunggul Hitam
11. Simpang GIA
12. Asrama Haji Padang

13. Perumahan Monang Indah
14. SMP Negeri 15 Padang
15. Fakultas Kedokteran Universitas Negeri Padang
16. Terminal Anak Air
17. Kayu Kalek
18. Kompleks Mega Permai

2. Koridor 2

1. RTH Imam Bonjol Padang
2. Pasar Gaung
3. Toko Abd Gani
4. Masjid Al Qahaar
5. Pabrik Karet Sungai Beremas
6. Pantai Nirwana
7. Kantor VTS Bungus
8. Pisang Salay Citra Minang
9. SDN 06 Cindakir
10. Cecen Cellular
11. Naldi Cell/Kedai Aura
12. Puskesmas Pembantu Bungus
13. SDN 12 Teluk Kabung
14. Simpang Sungai Pisang

3. Koridor 3

1. Terminal Anak Air
2. Kantor Camat Kuranji
3. Simpang Kalumbuk
4. Polsek/Puskesmas Kuranji
5. Utama Furniture
6. Poliklinik Fitria
7. Pool ALS
8. Simpang Lubeg
9. Simpang Beringin
10. Pampangan
11. Lantamal

catatan (9 juni):

- HARUS BISA MENYIMPAN DAN EMBACA DARI FILE
- TAMBAHIN FUNGSI BERISIAN HALTE PERKORIDOR
- PERPINDAHAN KORIDOR HANYA BOLEH 1 HALTER YANG SAMA
- PROSES PEMBENTUKAN TREE DINAMIS TERGANTUNG DI FILE
- HARUS ADA FUNGSI BUAT NENTUIN BAHWA PERKORIDOR SUDAH ADA HALTER YANG TERHUBUNG MAKSIMAL 1
- cara kerja breadth first search
- didokumen pakai data DUMMY bagian abstraksi data kalau program tetap data real yang ada di file

- menjelaskan output 1 per 1 di penjelasan output (pakai data dummy)
- penjelasan input menggunakan real data
- output rute
- di struktur data ADT apa saja yang digunakan (stack and queue digunakan untuk apa)
- yang gdi draw.io pertama buat generic
- spesifikasi proses, menjelaskan usulan program
 - menjelaskan pembentukan tree (mulai dari root dll)
 - menjelaskan menelusuri halte / pencarian halte (yang bfs)
 - validasi bahwa koridor memang terdaftar di file
 - BFS dipakai dimana
 - BFS boleh pakai punya orang
- abstraksi data ada visualisasi per struct jangan cuma bulet
- membuat flowchart alur proses dari spesifikasi proses dalam satu flow start to end, (butuh daftar modul, body/algorithm modul)
- UI (rancangan antarmuka)
-